

Technical Q&A; Question Bank

Coding, architecture, technology-stack, security, AI, and demo questions with concise model answers. Designed for judge discussions and team preparation.

Platform Evidence-backed developer credentials	Audience Judges, reviewers, recruiters, and team members
Stack Next.js, React, Prisma, SQLite, NextAuth, GitHub, Gemini, MediaPipe	Use Practice spoken answers and demonstrate the relevant screen or code path

How to use this pack: Keep answers evidence-based. Where a question refers to a feature, open the matching CodeVeritas screen or code path. The platform uses AI to support review, but it does not make a final hiring decision.

1. Product and architecture

Questions that explain the platform purpose, user flow, and evidence model.

QUESTION 01

What problem does CodeVeritas solve?

Answer: It helps developers demonstrate practical engineering evidence beyond a resume or profile claim. A challenge can link a public repository, recorded work, a repository-grounded PRI quiz, a speaking assessment, and independent reviews into one inspectable credential.

QUESTION 02

Who are the main user roles?

Answer: Developers submit work and complete assessments. Reviewers inspect evidence and provide independent rubric scores. Recruiters create challenges, discover candidates, compare submissions, and schedule interview invitations.

QUESTION 03

Walk through a typical challenge submission.

Answer: A recruiter creates a role-specific challenge. A developer submits a public repository or completes a recorded session. GitHub and configured Gemini analysis collect evidence, PRI and speaking assessments can be completed, reviewers score the work, and the verified result becomes visible in the developer's CodePassport.

QUESTION 04

Why is evidence more useful than activity counts alone?

Answer: Commit counts and profile claims do not explain quality, context, or trade-offs. CodeVeritas combines multiple signals and keeps their source visible, so a reviewer can inspect the repository, assessment status, and human feedback.

QUESTION 05

Does CodeVeritas claim to detect AI authorship?

Answer: No. AI tools are allowed. The system evaluates observable work evidence and does not claim to prove who authored every line of code or make a final hiring decision.

QUESTION 06

What happens if an external provider is unavailable during a recorded-session submission?

Answer: The submission and recording are persisted before network-dependent analysis starts. The user can retry analysis later rather than losing completed work. Recruiter comparison also has a labeled local evidence fallback if Gemini is unavailable.

2. Technology stack

Questions about the exact technologies used by the current CodeVeritas implementation.

QUESTION 07

Why did you choose Next.js 14?

Answer: Next.js lets the team build React user interfaces and authenticated API routes in one application. This keeps the dashboard, role checks, and server-side integration points close to the domain code.

QUESTION 08

What does React handle in this project?

Answer: React renders the developer, reviewer, and recruiter dashboards. Client components manage interactive state such as selected candidates, quiz progression, recording controls, modal workflows, and UI feedback.

QUESTION 09

How are users authenticated and role-scoped?

Answer: NextAuth uses credential-based sign-in and JWT sessions. Server routes call the current-user helper and check the role before accessing student, reviewer, or recruiter operations.

QUESTION 10

Why use Prisma?

Answer: Prisma gives the application typed, centralized access to users, challenges, credentials, reviews, work sessions, recordings, and interview invitations. It also keeps relations and uniqueness constraints explicit in the schema.

QUESTION 11

Which database is used now, and what is the production consideration?

Answer: The current local configuration uses SQLite for fast MVP setup. The repository contains a PostgreSQL migration artifact, but production deployment should reconcile the database configuration and use durable storage before claiming a hosted rollout.

QUESTION 12

How is GitHub integrated?

Answer: The server uses GitHub REST API calls to validate public repositories, read commit metadata, inspect timelines, and fetch bounded source files for repository-grounded questions. The code uses a server-side token when configured and never exposes it to the browser.

QUESTION 13

How is Gemini used?

Answer: Gemini generates configured repository, recorded-session, quiz, speaking, and comparison analysis. Outputs are bounded and validated before use, and they support human review rather than making an automated employment decision.

QUESTION 14

Why is MediaPipe Tasks Vision used?

Answer: MediaPipe runs face-presence and face-count checks in the browser during monitored assessments. It is used as an integrity signal, not for identity recognition or demographic inference.

3. Coding and implementation

Questions with code-level answers that are useful when judges ask how a feature works.

QUESTION 15

How do you prevent a recruiter from comparing the same developer twice?

Answer: The client enables comparison only when exactly two selected submission rows belong to distinct user IDs. The API repeats the same validation, so a crafted browser request cannot bypass the rule.

```
const canCompare = selectedRows.length === 2
  && new Set(selectedRows.map(row => row.userId)).size === 2;
```

QUESTION 16

How is the overall challenge score calculated?

Answer: The score is the equal-weight average of available assessment categories: recorded-session evidence, GitHub review, PRI quiz, and speaking. Pending dimensions remain null and are not treated as zero.

```
const available = assessmentKeys
  .map(key => scores[key])
  .filter(score => score !== null);
const overall = available.length
  ? Math.round(available.reduce((a, b) => a + b, 0) / available.length)
  : null;
```

QUESTION 17

How do you rank submissions fairly when scores are incomplete?

Answer: Rows with a valid overall score are ordered highest first. Rows with no overall score are placed after scored rows. Ties use submission time and then candidate name for a stable, reproducible ordering.

QUESTION 18

How do you validate a submitted GitHub URL?

Answer: The server parses the URL and requires HTTPS, the github.com hostname, and an owner/repository path. It then checks the repository through GitHub's API and rejects private or missing repositories.

QUESTION 19

How do you keep AI output from breaking the UI?

Answer: The server asks Gemini for JSON with a response schema, parses and validates required fields, bounds text lengths, and uses fallback paths when an external response is incomplete. The UI only renders validated comparison arrays and scores.

QUESTION 20

How do you avoid losing a recorded session when analysis fails?

Answer: The work-session completion route first persists a credential and links the saved recording. Only after that does it contact GitHub and Gemini. A later failure is stored as an analysis error and can be retried.

QUESTION 21

Why do API routes repeat validation that the UI already performs?

Answer: Browser controls improve usability, but users can modify requests. Server routes are the security boundary, so they validate authentication, roles, ownership, identifiers, repository format, and state transitions again.

QUESTION 22

How do you protect secrets in the application?

Answer: Server credentials such as GEMINI_API_KEY, GITHUB_TOKEN, and NEXTAUTH_SECRET are read from environment variables. API keys stay in server route headers and are never returned by a client endpoint or included in browser code.

4. Reliability, privacy, and demo questions

Questions judges often ask after the primary technical walkthrough.

QUESTION 23

What are the main external dependencies, and how do you handle failures?

Answer: The key external dependencies are GitHub, Gemini, and browser media capabilities. The application uses timeouts, clear user-facing errors, model fallbacks where configured, retry paths for saved submissions, and a local evidence comparison fallback for the recruiter view.

QUESTION 24

What data is sent to Gemini?

Answer: Only the bounded evidence required for the configured analysis is sent from the server, such as public repository snippets and consented recording evidence. The platform does not expose raw private answers in public CodePassports.

QUESTION 25

What does a public CodePassport show?

Answer: It presents developer identity information chosen for the profile, verified challenge credentials, available scores, and evidence-derived portfolio analysis. It excludes raw recordings and private assessment answers.

QUESTION 26

What is the current scaling limitation?

Answer: The first likely bottlenecks are reviewer throughput, GitHub quotas, and AI-provider quotas. The MVP uses local recording storage in development, so a production system needs durable private object storage, queues, retries, and retention controls.

QUESTION 27

How would you demonstrate the project to a judge in two minutes?

Answer: Start with a recruiter-created challenge, show a developer submission and its score breakdown, open repository or recording evidence, show the reviewer decision path, then open CodePassport and recruiter comparison. State that AI assists evidence analysis while a human keeps the final decision.

QUESTION 28

What is the strongest honest limitation to mention?

Answer: The product is an MVP with provider and network dependencies. It provides evidence signals and review workflows, not proof of authorship, code correctness, or job performance. A production version needs a hosted database, durable recording storage, job queues, and operational monitoring.

Practice tip: Use these answers as a guide, then explain the exact screen or code path you can demonstrate. Avoid claiming behavior that is not in the running application.